

# Database Assessment: Database Schema Design for Social Media Platform

## Requirements

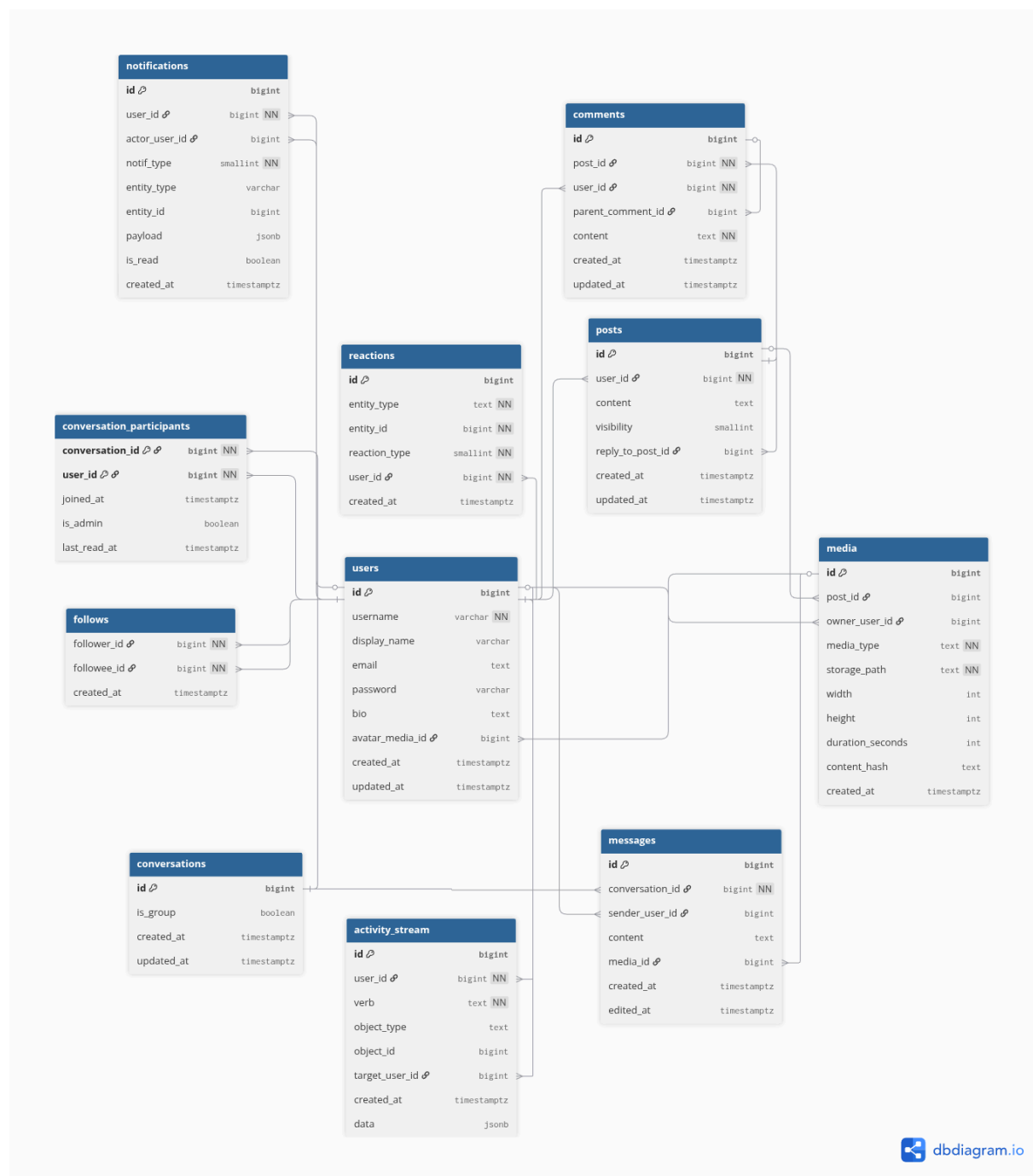
- User profiles and relationships (followers/following)
- Posts with multimedia content
- Comments and reactions
- Private messaging system
- Activity feeds and notifications

## Database Schema and ERD

Based on requirements, the table below shows entities that are needed.

| Table                     | Description                                  |
|---------------------------|----------------------------------------------|
| users                     | Stores user profiles                         |
| follows                   | Stores follower/followee relationships       |
| posts                     | User-generated posts, text + optional media  |
| users                     | Stores user profiles                         |
| media                     | Metadata and storage path of uploaded media. |
| comments                  | User comments on posts                       |
| reactions                 | Likes or reactions on posts/comments         |
| conversations             | Private/group chat container                 |
| conversation_participants | Members of a conversation                    |
| messages                  | Individual messages in a conversation        |
| notifications             | Stores notifications sent to users           |
| activity_streams          | Generalized activity log (feed).             |

## ERD



## Relation and Constraint

- **follows**: composite PK (**follower\_id**, **followee\_id**) prevents follow duplication. **ON DELETE CASCADE** used so it would be clear if the user was deleted.
- **posts.user\_id** FK to **users**.
- **media.post\_id** optional so single media may belong to post; **owner\_user\_id** for avatar or media uploaded but not attached yet.
- **reactions** unique so the user wont spam the same reaction entity.

- `conversation_participants.last_read_at` untuk hitung unread messages.

## Indexing Strategy

Indexes are designed for common queries such as feed generation, comments lookup, reaction counting, and unread messages. Composite indexes support WHERE + ORDER BY patterns.

```
-- posts: query by user + order by created_at
CREATE INDEX idx_posts_user_created_at ON posts(user_id, created_at DESC);

-- posts global: for retrieving recent public posts
CREATE INDEX idx_posts_created_at ON posts(created_at DESC);

-- media by post
CREATE INDEX idx_media_post_id ON media(post_id);

-- comments: by post for list
CREATE INDEX idx_comments_post_created ON comments(post_id, created_at
DESC);

-- reactions: to count reactions per entity
CREATE INDEX idx_reactions_entity ON reactions(entity_type, entity_id,
reaction_type);

-- follows: to get followers/ followees quickly
CREATE INDEX idx_follows_followee ON follows(followee_id, created_at);
CREATE INDEX idx_follows_follower ON follows(follower_id, created_at);

-- conversations participants
CREATE INDEX idx_conv_user ON conversation_participants(user_id,
last_read_at);

-- messages by conversation (to page messages)
CREATE INDEX idx_messages_conv_created ON messages(conversation_id,
created_at DESC);

-- notifications
CREATE INDEX idx_notifs_user_unread ON notifications(user_id, is_read,
created_at DESC);

-- activity_stream: query by actor or by created_at
CREATE INDEX idx_activity_user_created ON activity_stream(user_id,
created_at DESC);
```

## Sample SQL Queries for Feed Generation

SQL for Basic feed, that are posts from users the current user follows

```
-- Parameters: :current_user_id, :last_created_at, :last_id, :limit
SELECT p.*,
       u.username, u.display_name,
```

```

        COALESCE(r.count_like,0) AS likes_count,
        COALESCE(c.count_comment,0) AS comments_count
FROM posts p
JOIN users u ON u.id = p.user_id
-- only posts from people I follow (plus my own posts)
WHERE p.user_id IN (
    SELECT followee_id FROM follows WHERE follower_id = :current_user_id
    UNION
    SELECT :current_user_id
)
AND (p.created_at, p.id) < (:last_created_at, :last_id) -- keyset
pagination, pass last row's values
AND p.visibility = 1 -- or apply visibility checks
ORDER BY p.created_at DESC, p.id DESC
LIMIT :limit;

-- Pre-join aggregates using lateral or subqueries:
LEFT JOIN (
    SELECT entity_id, COUNT(*) FILTER (WHERE reaction_type = 1) AS count_like
    FROM reactions WHERE entity_type='post' GROUP BY entity_id
) r ON r.entity_id = p.id
LEFT JOIN (
    SELECT post_id, COUNT(*) AS count_comment FROM comments GROUP BY post_id
) c ON c.post_id = p.id;

```

## Caching Strategy

Recommended caching strategy:

- Use CDN for serving media files: Store media in object storage (S3) and serve via CDN (CloudFront/Cloudflare). Media metadata in DB; URLs are CDN-hosted.
- Redis for user timelines, counters, and unread message counts.
- Materialized views or batch jobs for personalized feed ranking.
- Use WebSocket/Socket.IO (or push via gateway) to send notifications. Use Redis pub/sub or Kafka as central event bus.
- Hybrid fan-out strategy: fan-out (push on write) for normal users (followers < X (e.g., 5000), pull model for celebrity accounts (pull when needed, merge top celebrity posts on feed read).
- **Cache invalidation & consistency**
  - On write (new post/comment/reaction):
    - Update DB.
    - Publish message on message broker (Kafka/Rabbit/NSQ) or Redis Pub/Sub.
    - Consumers update Redis caches: timeline sorted-sets, counters, cached post payloads.
    - For reactions/comments counters: increment Redis then background worker persists to DB periodically (batching).
  - For deletes/edits: publish events to invalidate caches and update DB.